



SaGD: A Node-Level Differentially Private Graph Learning Framework with Sensitivity-Aware Gradient Descent

Jianxin Wei

National University of Singapore
Singapore, Singapore
jianxinwei@u.nus.edu

Xiaokui Xiao

National University of Singapore
Singapore, Singapore
xkxiao@nus.edu.sg

Ergute Bao

Mohamed bin Zayed University of Artificial Intelligence
Abu Dhabi, United Arab Emirates
ergute.bao@mbzuai.ac.ae

Ting Yu

Mohamed bin Zayed University of Artificial Intelligence
Abu Dhabi, United Arab Emirates
ting.yu@mbzuai.ac.ae

Abstract

Learning from graph-structured data is fundamental to many Web applications, such as recommendation systems and social network analysis. While neural networks achieve state-of-the-art performance in these tasks, training them on sensitive graph data poses privacy risks. Node-level differential privacy (node-DP) provides strong protection for individual users represented as nodes; however, achieving node-DP is challenging due to the intricate dependencies among interconnected nodes. These dependencies, in turn, complicate node-level sensitivity analysis, which is a key step in differentially private learning. To bound the sensitivity, existing approaches typically (i) rely on black-box per-sample gradient clipping, which often overestimates sensitivity and introduces an excessive amount of DP noise; and (ii) prune edges from nodes with high degrees, which leads to erroneous privacy proofs.

In this work, we introduce SaGD, a rigorous node-DP framework for graph learning. Our framework consists of two stages: (i) message propagation over the original graph, where we weight edges to control sensitivity without edge pruning, enabling tight, closed-form sensitivity bounds; and (ii) neural network training on the propagated messages using perturbed gradients with explicit mathematical derivations of sensitivity, without relying on gradient clipping. This sensitivity-aware design allows SaGD to add only the necessary amount of DP noise for rigorous privacy guarantees while achieving high computational efficiency. Extensive experiments show that SaGD achieves significantly higher model utility under the same privacy budgets and up to 10× training speedup.

CCS Concepts

• Security and privacy → Privacy protections; • Computing methodologies → Machine learning algorithms.

Keywords

Differential Privacy; Graph Learning

ACM Reference Format:

Jianxin Wei, Ergute Bao, Xiaokui Xiao, and Ting Yu. 2026. SaGD: A Node-Level Differentially Private Graph Learning Framework with Sensitivity-Aware Gradient Descent. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792223>

Resource Availability:

The source code of this paper has been made publicly available at <https://doi.org/10.5281/zenodo.18370749>.

1 Introduction

Graph data is fundamental to the Web, where social networks, e-commerce platforms, and knowledge bases can all be represented as graphs [25, 37]. Learning from such structures powers applications such as recommendation, search, and community detection. However, these Web graphs often contain highly sensitive information: node (Web users) attributes can reveal geographic locations or browsing interests, while edges (Web users' connections/communications) may disclose social ties, interaction patterns, or online trajectories, raising serious concerns under privacy regulations such as CCPA [15] and GDPR [20]. Trained graph models on such data are vulnerable to inference attacks that aim to recover node membership [33], attributes [22], or relationships [19]. These risks highlight the urgent need for rigorous privacy protection when learning from sensitive graph data. In this paper, we focus on developing a privacy-preserving graph learning algorithm.

Differential privacy (DP) is a gold standard for protecting individual information in data analysis. In graph settings, node-level DP (node-DP) [9] protects each node's features, connections, and even its presence. A typical procedure involves (i) assessing the sensitivity, which measures the maximum effect of adding/removing a node with its edges on outputs such as gradients, and (ii) adding calibrated noise according to the sensitivity to achieve privacy guarantees. While DP has been extensively studied in conventional machine learning [1, 34], adapting it to graphs is challenging because standard methods like DP-SGD [1] assume independent data, where one record does not influence others.

Key Challenge: Entangled Individuals. However, in graph data, nodes are interconnected, and information propagation in graph learning creates complex dependencies [23, 35]: each node iteratively aggregates information from its neighbors and updates its



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792223>

representation. As shown on the left of Figure 1, in the non-private (non-DP) framework, each network layer performs message propagation, linear transformation, and non-linear activation, and the resulting network gradients or outputs depend on all these components. In the worst case, a single node can connect to and influence the entire graph, and its effect is further amplified through the propagation of parameters and activations. This makes the sensitivity of model gradients or outputs particularly difficult to analyze, as one must bound the maximum impact of any node during training. While various node-DP methods have been proposed, they still struggle to balance rigorous privacy with model utility. We next outline the four main limitations of existing solutions.

1.1 Limitations of Existing Work

Limitation 1: Gradient Clipping. Although DP-SGD [1] works well for independent data (e.g., images), it faces a fundamental limitation in graph learning and often overestimates sensitivity. Specifically, DP-SGD clips each gradient's L_2 -norm to a fixed bound, making the iteration sensitivity proportional to the bound and the number of affected gradients due to adding/removing one record. For independent data, this corresponds to one gradient. For graph data, consider an example of adding one node to a fully connected graph of n existing nodes and connecting it to all of them. Then each node's neighborhood information changes by about $\frac{1}{n}$, as its degree increases from $n-1$ to n . Thus, the overall sensitivity of the graph is about $n \cdot \frac{1}{n} = 1$. However, DP-SGD assumes that all node gradients are affected, overestimating the true sensitivity by a factor of n . Existing methods for differentially private graph learning all build upon DP-SGD, inheriting this fundamental limitation.

Limitation 2: Edge Pruning or Sampling. Existing node-DP methods rely on edge pruning or sampling to control node sensitivity, which introduces theoretical gaps in their privacy guarantees. In particular, most of them [7, 8, 32, 42] employ edge pruning: they fix a degree threshold K , randomly drop edges from nodes exceeding K , and assume that adding/removing a node affects at most K neighbors. However, this assumption is invalid in theory, and edge pruning under node-DP remains non-trivial [4, 10]. For example, suppose graph G has all nodes with degree bounded by 1. If we insert a new node that connects to all existing nodes, we obtain G' . When using $K = 1$ as the degree threshold, no edge in G will be clipped. However, in G' , more than half of the edges will be dropped in expectation. This creates a large discrepancy between the two neighboring graphs after pruning, causing existing methods to fail node-DP. An additional example is provided in Appendix A.1, showing that the worst-case sensitivity is n even with $K = 1$.

Although other work [38] samples edges to reduce sensitivity instead of pruning, it also fails to ensure node-DP. In this method, the edge sampling probability is inversely proportional to the degrees of its endpoints. When a new node is added and connected to all other nodes, the sampling probability of every existing node may change, resulting in $O(n)$ sensitivity. However, this method only accounts for the change of the newly added node while ignoring the others, thereby underestimating the sensitivity [39].

Limitation 3: Combinatorial Explosion of Sensitivity. Even with an ideal pruning algorithm that limits each node to K edges (or when the graph itself enforces this), prior work still gives loose

sensitivity bounds of $O(K^m)$, where m is the propagation depth (also referred to as hops or steps). To reduce sensitivity, these methods prune heavily ($K \leq 4$) and restrict propagation to $m = 1$, drastically distorting the graph topology. Yet many real Web-related graphs have average degrees in the tens or hundreds, and tasks often require long-range information (e.g., ≥ 4 -hop [13, 21]).

Limitation 4: Message Propagation Distortion. As shown in the middle of Figure 1, prior works [7, 32, 42] inject substantial noise into the message propagation to realize node-DP, which greatly impairs the quality of graph information. Because message propagation involves large-scale matrix computations over all edges and nodes, injecting noise into these matrices markedly reduces their informativeness and degrades the utility of the resulting model.

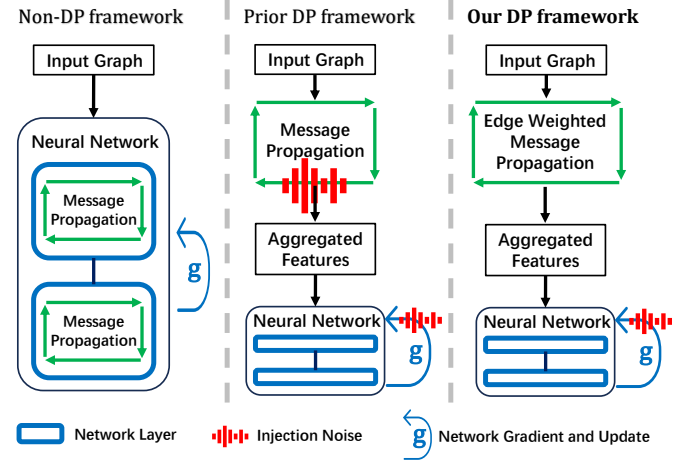


Figure 1: Comparison of learning frameworks: the non-private method, prior DP methods, and ours. Our design first performs message propagation on the input graph and then feeds the aggregated features into model training, where DP noise is injected only into the gradients.

1.2 Our Contributions

To address the challenge and limitations, we propose SaGD, a rigorous node-level differentially private graph learning framework, which is built on a Sensitivity-aware Gradient Descent algorithm. The main contributions of this work are summarized below.

Contribution 1: Decoupled Framework with Gradient-only Perturbation. We decouple message propagation from the neural network and propose a corresponding privacy-preserving mechanism that injects noise only into the training gradients. As shown on the right of Figure 1, we perform message propagation once at the beginning, without involving the network, and then analyze the sensitivity of the propagation results (i.e., aggregated features). The aggregated features are subsequently fed into the network, where we map their sensitivity to the gradient sensitivity and add calibrated noise to protect the published model. Unlike prior frameworks, our design makes sensitivity analysis more tractable and avoids injecting noise into the process or results of message propagation, which is the core of graph learning. This addresses **Key**

Challenge and Limitation 4. Moreover, decoupling message propagation from model training has been shown to achieve competitive performance in non-private graph learning [14, 18, 36], compared to coupled models.

Contribution 2: Edge-weighted Propagation with Closed-form Sensitivity Bounds. In the message propagation stage, we design an edge weighting mechanism to control the influence of each node, replacing prior pruning and sampling methods that discard many edges and rely on incorrect sensitivity assumptions. Our mechanism reduces the sensitivity caused by n edges (addressing **Limitation 2**), preserves the original graph topology, and also accelerates training by avoiding the need to traverse edges one-by-one during pruning. In addition, we provide *the first tight, closed-form sensitivity bounds* for the aggregated features at arbitrary propagation depth under node-DP. Our analysis reduces the sensitivity from $O(n)$ or $O(K^m)$ in prior work to $O(m)$, addressing **Limitation 3**. Furthermore, we incorporate a personalized PageRank (PPR) scheme [13] into message propagation, which has been shown to improve performance in non-private settings and enables us to reduce the sensitivity from $O(m)$ to $O(1)$.

Contribution 3: Gradient Perturbation without Clipping. Third, we propose a sensitivity-aware gradient perturbation algorithm that derives sensitivity directly from the model input and the current model state, eliminating the need for clipping. Our analysis precisely tracks each input's influence to its gradient, addressing **Limitation 1**. The gradient sensitivity can be derived purely from the sensitivity of the propagation results, without relying on the details of the propagation process, ensuring broad applicability. By avoiding per-sample clipping, our method also significantly accelerates training, overcoming a key bottleneck of DP-SGD-based methods identified in both academia [1, 6] and industry [41].

Finally, we evaluate SaGD on node classification tasks across multiple widely used real-world datasets under varying privacy budgets. The results demonstrate that SaGD consistently outperforms existing methods by a substantial margin in accuracy while achieving over 10 $\times$ faster training.

2 Preliminaries

For vectors we use $\|\cdot\|_2$ and $\|\cdot\|_1$ to denote their L_2 -norms and L_1 -norms, respectively. For matrices, $\|\cdot\|_2$ denotes the spectral norms.

2.1 Differential Privacy

Differential privacy (DP) offers strong protection for individuals' private data by limiting what an adversary can infer about the presence or absence of any individual. The standard definition is (ϵ, δ) -DP:

DEFINITION 1 ((ϵ, δ) -DP [11]). *A randomized algorithm $\mathcal{A} : \mathcal{D} \rightarrow \mathcal{R}$ satisfies (ϵ, δ) -DP if given any neighboring datasets $D, D' \subseteq \mathcal{D}$, we have*

$$\Pr[\mathcal{A}(D) \in O] \leq \exp(\epsilon) \cdot \Pr[\mathcal{A}(D') \in O] + \delta, \quad (1)$$

for any set of output $O \subseteq \mathcal{R}$.

Here, the (ϵ, δ) pair represents the *privacy budget*. The privacy budget effectively measures the indistinguishability between $\mathcal{A}(D)$

and $\mathcal{A}(D')$. Smaller ϵ and δ mean a smaller privacy budget and a stronger privacy protection, and vice versa. DP also requires defining *neighboring datasets*. In node-DP [32], we define node-level neighboring datasets as:

DEFINITION 2 (NODE-LEVEL NEIGHBORING DATASETS). *Two graph datasets D and D' are node-level neighboring if D' can be obtained from D by adding or removing a single node together with all its incident edges.*

An alternative privacy notion is Rényi DP (RDP) [27], which enables more precise tracking of cumulative privacy loss and thus refined analysis. For a given δ , an (α, τ) -RDP guarantee can be converted to (ϵ, δ) -DP using Theorem 21 in [3].

2.2 Non-private Graph Learning Models

In the non-private setting, many graph learning models have been developed, among which graph neural networks (GNNs) are the most widely used. We give a brief overview of representative GNN structures and the training pipeline.

Input. The input of a GNN is a graph $G = (V, E)$, where set V contains $n = |V|$ nodes and set E represents the edges connecting the nodes. Each node $i \in V$ is usually associated with a normalized feature vector $\mathbf{x}_i \in \mathbb{R}^d$ and a one-hot encoded label vector $\mathbf{y}_i \in \mathbb{R}^c$, where $\|\mathbf{x}_i\|_2 = 1$, d is the dimensionality of the features, and c is the number of classes. The node feature matrix and the label matrix are denoted as $\mathbf{X} \in \mathbb{R}^{n \times d}$ and $\mathbf{Y} \in \mathbb{R}^{n \times c}$, respectively. E is usually represented by an adjacency matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$, where A_{ij} is 1 if there is an edge $\langle i, j \rangle$ between nodes i and j ; and 0, otherwise.

Model. A typical GNN layer updates the representations of nodes via message propagation, also referred to as message passing or aggregation. In particular, it uses a normalized adjacency matrix

$$\bar{\mathbf{A}} = (\mathbf{D} + \mathbf{I})^{-1}(\mathbf{A} + \mathbf{I}), \quad (2)$$

and a weight matrix $\Theta^{(l)}$, where $\mathbf{I}$ is an identity matrix, and $\mathbf{D}$ is a diagonal matrix with the i -th diagonal element representing the degree of node i . The output $\mathbf{H}^{(l)}$ of the layer l is

$$\mathbf{H}^{(l)} = h\left(\bar{\mathbf{A}}\mathbf{H}^{(l-1)}\Theta^{(l)}\right), \quad (3)$$

where h is an activation function and $\mathbf{H}^{(0)} = \mathbf{X}$. The final output of a GNN containing m layers is denoted as $\hat{\mathbf{Y}} = \mathbf{H}^{(m)}$.

Training objective. The training process iteratively minimizes the error between the predicted $\hat{\mathbf{Y}}$ and the true labels $\mathbf{Y}$, quantified by some function F such as the softmax cross-entropy loss [5].

2.3 Personalized PageRank Propagation

Personalized PageRank (PPR) [13, 30] enhances propagation by introducing a restart probability that replaces part of the aggregated features with the originals, unlike standard propagation that repeatedly multiplies $\bar{\mathbf{A}}$. With $\mathbf{R}_0 = \mathbf{I}$ denoting the roots, the PPR propagation matrix at depth m is defined as

$$\bar{\mathbf{R}}_m = (1 - \gamma)\bar{\mathbf{A}}\bar{\mathbf{R}}_{m-1} + \gamma\mathbf{I} = \gamma \sum_{i=0}^{m-1} (1 - \gamma)^i \bar{\mathbf{A}}^i + (1 - \gamma)^m \bar{\mathbf{A}}_m, \quad (4)$$

where $\bar{\mathbf{A}}^i$ represents the product of i $\bar{\mathbf{A}}$ matrices, and $\gamma \in [0, 1]$ is the restart probability, controlling the balance between propagation

and resetting to the initial state. Note that when $\gamma = 0$, the PPR propagation reduces to the original propagation used in GNNs.

3 Problem Setting

Our goal is to design an effective node-DP algorithm for learning from sensitive graph data, ensuring that the released trained model does not leak the privacy of the *training data*. We adopt node-DP and focus on the node classification task following [8, 32, 38, 42]. For simplicity, we assume all edges are undirected, as directed edges are usually treated as undirected in practice.

Threat Model. We follow the standard threat model in the centralized DP setting [1]. The attacker observes the released model parameters after training, does not interfere with the training process, and attempts to infer information about a target node (including its features and edges) from the training graph. We adopt DP to protect against membership, edge, and feature inference attacks, which provably bounds the attacker’s ability to infer sensitive information about any individual node by (ϵ, δ) .

4 Our Solution: SaGD

4.1 Solution Overview

Our framework decouples message propagation from the linear transformation of node features in each layer of traditional non-private graph learning models. Accordingly, it consists of two main parts: message propagation on the graph and model training. The first part includes preprocessing, propagation, and propagation sensitivity, detailed in Sections 4.2.1–4.2.2. The second part includes prediction via the network architecture, gradient computation, and gradient perturbation, detailed in Sections 4.2.3–4.2.5.

In Section 4.2.1, we preprocess the adjacency matrix $\mathbf{A}$ to simplify the subsequent analysis of the sensitivity of the aggregated features. A conventional preprocessing for $\mathbf{A}$ is row-wise normalization (Equation (2)), but it is not suitable for sensitivity analysis. As introduced in our **Contribution 2**, we propose a novel edge weighting mechanism, formulated as an element-wise normalization of $\mathbf{A}$ (Equation (5)). It enables high-utility graph learning under rigorous node-DP, avoiding reliance on problematic edge pruning or sampling algorithms. We provide the intuition behind this sensitivity reduction in Section 4.2.1.

In Section 4.2.2, we present our propagation process, identical to non-private GNNs. Unlike prior methods that are restricted to 1-hop or inject heavy noise, we preserve the full topology to capture both short-range and long-range dependencies. To further enhance utility and reduce sensitivity, we adopt personalized PageRank (PPR) propagation, which reduces to standard GNN propagation when $\gamma = 0$. As part of our **Contribution 2**, we derive the first tight, closed-form sensitivity bounds for both standard and PPR propagation under conventional and our proposed normalizations (Lemma 1).

In Section 4.2.3, we describe the network architecture and prediction generation, followed by the loss, gradient computation, and gradient sensitivity analysis in Section 4.2.4. Section 4.2.5 introduces our gradient perturbation method (**Contribution 3**), where sensitivity is derived directly from formulas in terms of network parameters and feature sensitivity, without clipping; detailed proofs are in Section 5.1. Importantly, gradient sensitivity depends only on

input sensitivity rather than the propagation process, making our method suitable for graphs and other domains where one record may affect many, unlike traditional DP-SGD, which treats any feature change as a fully altered gradient.

As part of our **Contribution 3**, eliminating per-sample clipping greatly reduces overhead. Prior work [1, 6, 41] identifies per-sample gradient computation and clipping as the main bottleneck in DP-SGD. Our algorithm computes gradients in a fully vectorized way over the batch, rather than iterating over individual samples, achieving at least 10× faster training and higher accuracy. Detailed runtime comparisons are given in Tables 2 & 3.

4.2 Solution Details

4.2.1 Preprocessing. The input of our algorithm is a graph dataset $D = \langle V, E, \mathbf{X}, \mathbf{Y} \rangle$. We normalize each row of $\mathbf{X}$, i.e., each feature vector, to have an L_2 -norm of 1. We use an adjacency matrix $\mathbf{A}$ to represent the edges E , where $A_{ij} = A_{ji} = 1$ if there is an edge between nodes i and j , and 0 otherwise.

The conventional row-wise normalization of $\mathbf{A}$ (Equation (2)) leads to excessive sensitivity. In the worst case, adding a central node connected to n isolated nodes makes n rows of $\tilde{\mathbf{A}}$ change from identity rows to two $1/2$ entries, resulting in a total sensitivity of n . To reduce the sensitivity, we propose a novel edge weighting mechanism, referred to as element-wise normalization for $\mathbf{A}$. Formally, $\mathbf{A}$ is normalized to $\tilde{\mathbf{A}}$ through

$$\tilde{A}_{ij} = \begin{cases} \min\left(\frac{\mu}{K}, \frac{1}{k_i+1}\right), & A_{ij} = 1 \text{ \& } i \neq j, \\ 1 - \sum_{s=1}^n A_{is}, & i = j, \\ 0, & A_{ij} = 0 \text{ \& } i \neq j, \end{cases} \quad (5)$$

where μ is a hyperparameter, k_i is the degree of node i , and K is the maximum possible node degree. Note that the diagonal elements of $\tilde{\mathbf{A}}$ are positive, as proved in Appendix A.2. Without prior knowledge, K should be set to n . K can be optimized under the following conditions: (i) the graph enforces a structural constraint such that no node has more than $\tilde{K}$ edges; (ii) the privacy protection is relaxed from node-DP to $\tilde{K}$ -edge DP [7]; or (iii) an ideal pruning algorithm can ensure that adding/removing a node affects at most $\tilde{K}$ edges. In these cases, we can set $K = \tilde{K}$ instead of n to leverage more edge information without compromising privacy.

This edge weighting design enables our algorithm to meet the rigorous node-DP requirements. Under rigorous node-DP, one must consider the worst-case scenario in which two neighboring datasets differ by a node together with all of its neighboring nodes. Existing approaches typically rely on edge pruning or sampling to limit the number of neighbors affected by a node; however, as discussed in Section 1.1, such assumptions do not hold. Correcting these methods to satisfy rigorous node-DP would require substantially scaling the injected noise, which in turn leads to poor utility (see Table 1). Although edge weighting introduces disparities in the amount of information retained across nodes and degrades performance compared to non-private GNNs (Equation (2)), it substantially reduces sensitivity. Experimental results in Table 1 show that this reduction in sensitivity more than compensates for the induced disparities, leading to improved utility over other DP baselines.

4.2.2 Message Propagation. Given the element-wise normalized $\tilde{\mathbf{A}}$, we apply the PPR propagation scheme described in Equation (4) and construct our propagation matrix $\mathbf{R}_m$. Then we aggregate the features using $\mathbf{R}_m$, as follows

$$\mathbf{Z}_m = \mathbf{R}_m \mathbf{X}. \quad (6)$$

We use $\mathbf{R}_m, \mathbf{Z}_m$ for results computed with $\tilde{\mathbf{A}}$ (element-wise normalized $\mathbf{A}$) by Equations (4, 6), and $\bar{\mathbf{R}}_m, \bar{\mathbf{Z}}_m$ for those with $\bar{\mathbf{A}}$ (row-wise normalized $\mathbf{A}$). To facilitate the subsequent analysis of the gradient sensitivity, we define a metric $\psi(\mathbf{Z}, \mathbf{Z}')$ in Definition 3 to evaluate the sensitivity of the aggregated features $\mathbf{Z}$ when a node and its edges are added or removed. The aggregated feature of the added or removed node is excluded, since its sensitivity can be analyzed separately and is straightforward. The sensitivities of standard propagation $\tilde{\mathbf{A}}_m \mathbf{X} / \bar{\mathbf{A}}_m \mathbf{X}$ and PPR propagation $\mathbf{Z}_m / \bar{\mathbf{Z}}_m$, are stated in Lemma 1, with detailed proofs in Appendix B.

DEFINITION 3 (SENSITIVITY METRIC). $\mathbf{Z}$ and $\mathbf{Z}'$ are the aggregated features on D and any of its neighboring datasets D' , respectively, computed by the same aggregation method (e.g., by Equation (6)). Let $\mathbf{z}_i$ and $\mathbf{z}'_i$ denote the i -th rows of $\mathbf{Z}$ and $\mathbf{Z}'$, respectively. The variation of $\mathbf{Z}$ compared to $\mathbf{Z}'$, excluding the added/removed node, is defined as

$$\psi(\mathbf{Z}, \mathbf{Z}') = \psi(\mathbf{Z}', \mathbf{Z}) \triangleq \sum_{i \in V \cap V'} \|\mathbf{z}_i - \mathbf{z}'_i\|_2.$$

The sensitivity of $\mathbf{Z}$ is defined by $S(\mathbf{Z}) = \sup_{D, D'} \psi(\mathbf{Z}, \mathbf{Z}')$. $\square$

LEMMA 1. The sensitivities $S(\cdot)$ of $\tilde{\mathbf{A}}_m \mathbf{X}$ and $\mathbf{Z}_m$ are given by

$$S(\tilde{\mathbf{A}}_m \mathbf{X}) = 2\mu m; S(\mathbf{Z}_m) = \frac{2\mu(1-\gamma)}{\gamma} [1 - (1-\gamma)^m].$$

If the influenced edges of any node can be bounded by K , then the sensitivities $S(\cdot)$ of $\bar{\mathbf{A}}_m \mathbf{X}$ and $\bar{\mathbf{Z}}_m$ are given by

$$S(\bar{\mathbf{A}}_m \mathbf{X}) = Km; S(\bar{\mathbf{Z}}_m) = \frac{K(1-\gamma)}{\gamma} [1 - (1-\gamma)^m]. \quad \square$$

4.2.3 Network Architecture. We process the aggregated features from Equation (6) using an ω -layer network with weights $\Theta = \{\Theta_1, \dots, \Theta_\omega\}$ to generate predictions $\hat{\mathbf{Y}}^\top = (\hat{y}_1, \dots, \hat{y}_n)$. Formally, given the input $\mathbf{H}_0 = (\mathbf{z}_1, \dots, \mathbf{z}_n)$, the network is defined as

$$\mathbf{H}_l = h(\Theta_l \mathbf{H}_{l-1}), l \in [1, \omega - 1]; \quad \hat{\mathbf{Y}}^\top = \Theta_\omega \mathbf{H}_{\omega-1}. \quad (7)$$

where h is the activation function applied in the intermediate layers. For h , we consider those with bounded first-order and second-order derivatives, such as GELU and Mish [28], widely used variants related to ReLU and SELU.

4.2.4 Loss & Gradient. We iteratively update the network parameters via gradient descent. In each iteration, the gradient is computed through a loss function F that measures the distance between the network output $\hat{\mathbf{Y}}$ and the true labels $\mathbf{Y}$. Formally, the general form of F is defined as $F(\hat{\mathbf{Y}}, \mathbf{Y}) = \sum_{i=1}^n f(\hat{y}_i, y_i)$, where function f evaluates the distance between $\hat{y}_i$ and y_i . Let y_{ij} and $\hat{y}_{ij}$ be the j -th element of y_i and $\hat{y}_i$, respectively. For the function f , we consider those with bounded first-order and second-order derivatives, such as the commonly used softmax cross-entropy loss [5] and pseudo-huber loss [17], detailed in Appendix C.2.

The gradient $\mathbf{g}$ is computed by $\mathbf{g} = \nabla_{\Theta} F(\hat{\mathbf{Y}}, \mathbf{Y})$. For the sensitivity of gradient sum $\mathbf{g}$, denoted $S(\mathbf{g})$, its exact form depends on

the specific model architecture and the corresponding gradient computation. We illustrate the derivation using a two-layer network, following the model used in prior work [24, 32, 35, 42], as a representative example. We present the results in Lemma 2, with detailed analysis and proof provided in Section 5.1 and Appendix C.1, respectively. Derivations for networks with different numbers of layers follow the same procedure.

LEMMA 2. Let c_1 and c_2 denote the Lipschitz constants of the loss function f and its derivative, respectively. Let c_{h1} and c_{h2} denote the Lipschitz constants of the activation h and its derivative, respectively. Let $\theta_1 = \|\Theta_1\|_2$ and $\theta_2 = \|\Theta_2\|_2$. Based on the sensitivity $S(\mathbf{Z})$ of the input aggregated features $\mathbf{Z}$ (e.g., computed by Lemma 1), the gradient sensitivity of a two-layer network is

$$S(\mathbf{g}) = \sqrt{\sum_{l=1}^2 (S(g_{\text{new}}(\Theta_l)) + S(g_{\text{rest}}(\Theta_l)))^2},$$

where

$$\begin{aligned} S(g_{\text{new}}(\Theta_1)) &= c_1 c_{h1} \theta_2, \quad S(g_{\text{new}}(\Theta_2)) = c_1 \theta_1, \\ S(g_{\text{rest}}(\Theta_1)) &= (c_2 c_{h1}^2 \theta_2^2 \theta_1 + c_1 c_{h2} \theta_2 + c_1 c_{h2} \theta_2 \theta_1) S(\mathbf{Z}), \\ S(g_{\text{rest}}(\Theta_2)) &= (c_2 c_{h1} \theta_2 \theta_1^2 + c_1 c_{h1} \theta_1) S(\mathbf{Z}). \end{aligned} \quad (8)$$

In addition, for a one-layer network, the gradient sensitivity is

$$S(\mathbf{g}) = S(g_{\text{new}}(\Theta_1)) + S(g_{\text{rest}}(\Theta_1)) = c_1 + (c_1 + c_2 \theta_1) S(\mathbf{Z}). \quad \square$$

In Lemma 2, $S(g_{\text{new}})$ corresponds to the gradient sensitivity of the added or removed node between neighboring datasets, while $S(g_{\text{rest}})$ corresponds to the gradient sensitivity of the other nodes affected by the added or removed edges. Note that the Lipschitz constants c_1, c_2, c_{h1} , and c_{h2} are independent of the model inputs. Their values depend solely on the loss function f and the activation function h , which may vary case by case. The specific values of these Lipschitz constants used in this work are provided in Appendix C.2.

4.2.5 Gradient Perturbation and Model Update. Next, we introduce our perturbation on $\mathbf{g}$ to preserve its privacy. We add Gaussian noise, proportional to the sensitivity $S(\mathbf{g})$ and a noise multiplier σ_T , to $\mathbf{g}$, as follows.

$$\tilde{\mathbf{g}} = \mathbf{g} + \mathcal{N}\left(\mathbf{0}, (S(\mathbf{g})\sigma_T)^2 \mathbf{I}\right), \quad (9)$$

where $\mathcal{N}$ refers to the Gaussian distribution and σ_T is an adjustable parameter used to match the privacy constraint ϵ and δ . The detailed computation and intuition behind $S(\mathbf{g})$ are deferred to Section 5.1.

At the end of each training iteration, we update the network parameters Θ using the noisy gradient $\tilde{\mathbf{g}}$, as $\Theta \leftarrow \Theta - \eta \cdot \frac{\tilde{\mathbf{g}}}{\tilde{n}}$, where η is a tunable learning rate and $\tilde{\mathbf{g}}$ is averaged by $\tilde{n}$. Here, $\tilde{n}$ denotes the perturbed node count. Since adding/removing a node changes the total number of nodes and may leak privacy, we protect n by adding Gaussian noise: $\tilde{n} = n + \mathcal{N}(\mathbf{0}, \sigma_n^2)$, where σ_n is a noise multiplier determined by the privacy budget allocated to protect n .

4.3 Complete Algorithm

The complete SaGD algorithm is summarized in Algorithm 1. Line 1 preprocesses the node features $\mathbf{X}$ and normalizes each feature to have an L_2 -norm bounded by 1. Line 2 successively compute the

Algorithm 1: SaGD

Input: Training dataset $D = \langle V, E, X, Y \rangle$, loss function F , restart probability γ , noise multipliers σ_n and σ_T , training iteration T , learning rate η .

Output: Network parameters Θ .

- 1 Normalize each row of X ;
- 2 Compute $\tilde{A}$, R_m , and Z by Equations (5, 4, 6), respectively;
- 3 **for** $i \in [1, T]$ **do**
- 4 Compute $\hat{Y}$ by Equation (7) and $g = \nabla_{\Theta} F(\hat{Y}, Y)$;
- 5 Compute $S(Z)$ and $S(g)$ by Lemmas 1 & 2, respectively;
- 6 $\tilde{g} = g + \mathcal{N}(\mathbf{0}, (S(g)\sigma_T)^2 \mathbf{I})$;
- 7 $\Theta \leftarrow \Theta - \eta \cdot \tilde{g} / \tilde{n}$, where $\tilde{n} = n + \mathcal{N}(\mathbf{0}, \sigma_n^2)$;
- 8 **return** Θ ;

normalized adjacency matrix $\tilde{A}$, propagation matrix R_m , and the aggregated features Z following Equations (5, 4, 6), respectively. Line 4 generates predictions $\hat{Y}$ for each node following Equation (7) and then computes the gradient g . Line 5 computes the sensitivity of the aggregated features Z and the gradient g . Lines 6–7 add noise proportional to $S(g)$ (computed from Lemma 2) and σ_T to g , and update Θ using the noisy gradient $\tilde{g}$. After T training iterations, the algorithm outputs the optimized Θ .

5 Analysis

The privacy guarantee of our solution, Algorithm 1, is provided in Theorem 1, with the proof detailed in Appendix C.3. The RDP guarantee in Theorem 1 can be converted to traditional (ϵ, δ) -DP following Theorem 21 in [3]. In practice, given privacy constraints ϵ and δ , we fix σ_n and select the minimal σ_T based on the remaining privacy budget.

THEOREM 1. *Algorithm 1 satisfies $(\alpha, \frac{T\alpha}{2\sigma_T^2} + \frac{\alpha}{2\sigma_n^2})$ -RDP.*

In the following, we present the gradient sensitivity analysis. For a neighboring dataset D' differing from D by adding (or removing) one node, we use ι to denote results on D' (e.g., Z' for the aggregated features and g' for the gradient). As shown in Section 4.2, we add calibrated noise with standard deviation proportional to $S(g)$ to the gradient sum g in Equation (9). Therefore, the key to achieving both privacy protection and high model utility lies in determining an appropriate value for $S(g)$.

5.1 Sensitivity Analysis of the Gradient g

Since the overall gradient is composed of the contributions from each layer, we analyze them individually and then combine the results. For layer Θ_l , the gradient difference between neighboring datasets consists of two parts: gradient $g_{\text{new}}(\Theta_l)$ from the added/removed node and gradient difference $g_{\text{rest}}(\Theta_l)$ from the other nodes.

Specifically, for the gradient of Θ_l for each prediction $\hat{y}_i$, denoted as $g(\Theta_l, \hat{y}_i)$, we have $g(\Theta_l, \hat{y}_i) = \frac{\partial f(\hat{y}_i, y_i)}{\partial \hat{y}_i} \cdot \frac{\partial \hat{y}_i}{\partial \Theta_l}$. For a new added node with aggregated feature z_{n+1} , $\|\frac{\partial f(\hat{y}_{n+1}, y_{n+1})}{\partial \hat{y}_{n+1}}\|_2$ can be bounded by a product of the Lipschitz constant of f , the current network parameters $\{\Theta\}_{l=1}^\omega$, and $\|z_{n+1}\|_2$. $\|\frac{\partial \hat{y}_{n+1}}{\partial \Theta_l}\|_2$ can be

bounded by $\{\Theta\}_{l=1}^\omega$ and $\|z_{n+1}\|_2$. Similarly, for each of the rest nodes, $\|\frac{\partial f(\hat{y}_i, y_i)}{\partial \hat{y}_i} - \frac{\partial f(\hat{y}_i', y_i')}{\partial \hat{y}_i'}\|_2$ can be bounded by the Lipschitz constant of f , $\{\Theta\}_{l=1}^\omega$, and $\|z_i - z_i'\|_2$. $\|\frac{\partial \hat{y}_i}{\partial \Theta_l} - \frac{\partial \hat{y}_i'}{\partial \Theta_l}\|_2$ can be bounded by $\{\Theta\}_{l=1}^\omega$ and $\|z_i - z_i'\|_2$. Hence, the change of the gradient sum of rest nodes, $g_{\text{rest}}(\Theta_l)$, can be bounded by $\sum_i^n \|z_i - z_i'\|_2 \leq S(Z)$, as established in Lemma 1, together with other known quantities such as the network parameter values. By summarizing the above derivations, we can obtain the sensitivity $S(g(\Theta_l))$ and $S(g)$. The exact form of $S(g)$ depends on the specific model architecture and the corresponding gradient computation, and its derivation follows the same procedure as in Lemma 2 and Appendix C.1.

We clarify that using the current network parameters $\{\Theta\}_{l=1}^\omega$ to compute $S(g)$ does not reveal private information. $\{\Theta\}_{l=1}^\omega$ are the outputs from the previous iteration. Before training, the initial network parameter values are random and non-private information. During each training iteration, the gradient is privatized by adding noise before being released. At the end of each iteration, $\{\Theta\}_{l=1}^\omega$ is updated using the noisy gradient. This is considered post-processing [12] that preserves the privacy guarantee. Consequently, the parameters obtained after each iteration can be directly used.

In DP-SGD-based methods, each node's gradient L_2 -norm across layers is compared with a clipping threshold, and all gradients are scaled if it is exceeded. In contrast, we derive the overall gradient sensitivity analytically without per-sample clipping, accelerate training (Table 2), and better preserve gradient directions.

6 Experiments

6.1 Settings

Baselines. We compare our SaGD algorithm against four private baselines, GAP [32], DPAR [42], DP-GNN [8], and DP-SGD [1], as well as one non-private baseline, GNN (Non-DP) [26]. DP-SGD refers to a straightforward implementation of DP-SGD [1] for training a multi-layer perceptron model, which uses only node features and labels as input, without incorporating any edge information. GNN (Non-DP) represents an upper limit of the private model's performance. We further include GNN (EW), which is GNN (Non-DP) equipped with our edge weighting, to evaluate the effect of this mechanism. We do not include DPDGC [7] in the comparison because it employs a newly defined privacy guarantee that is not widely accepted and is less stringent than those of other methods. In addition, we exclude [38], as subsequent analysis via [2, 39] revealed gaps that invalidate its claimed privacy guarantee.

Datasets. Following previous work [24, 40, 42], we evaluate four popular graph datasets, whose key statistics are summarized in Table 5 in Appendix D. For each dataset, we randomly split the nodes into 60%/20%/20% for training, validation, and test sets, respectively, following [16, 21]. The training set is treated as private, while the validation and test sets are public, in accordance with the competitors. We evaluate the classification accuracy of the solutions on the datasets by the micro-averaged F1 score.

Hyperparameters. We follow prior works [7, 8, 32, 42] and adopt strong privacy protection. We set $\epsilon \in \{4, 8\}$ and fix $\delta = \frac{1}{n+1}$. We allocate the privacy budget evenly across training iterations. Other hyperparameters are detailed in Appendix D.

Table 1: Classification accuracy (in percentage) of algorithms.

Dataset	Privacy budget	SaGD	GAP	DPAR	DP-GNN	DP-SGD	GNN(EW)	GNN(Non-DP)
Reddit	$\epsilon = 4$	89.8	39.8	12.4	9.2	67.7	90.8	95.0
	$\epsilon = 8$	90.3	41.7	12.4	10.7	70.2		
Facebook	$\epsilon = 4$	81.0	72.3	42.8	52.1	74.4	84.6	93.0
	$\epsilon = 8$	81.5	72.5	43.4	58.9	74.7		
Cora-ML	$\epsilon = 4$	65.2	33.6	27.3	16.8	62.6	86.2	89.3
	$\epsilon = 8$	74.0	40.7	28.0	16.8	66.8		
PubMed	$\epsilon = 4$	85.4	83.4	50.9	31.1	84.8	86.8	87.6
	$\epsilon = 8$	86.3	84.7	51.2	33.4	85.6		

6.2 Results

Classification Accuracy. The classification accuracy of all methods on the datasets under two privacy budgets is presented in Table 1. All methods are evaluated under the rigorous node-DP setting. Each experiment is repeated five times. As shown in Table 1, our solution SaGD consistently outperforms the competitors across all configurations. The superior performance of SaGD can be attributed to its algorithm design and privacy analysis. SaGD preserves the original message propagation, perturbs only the gradient, and injects substantially less noise for privacy preserving.

GAP suffers from **Limitations 1, 2, & 4** mentioned in Section 1.1. As shown in Table 1, GAP performs worse than SaGD, and even worse than DP-SGD, for two reasons. First, GAP injects substantial noise into the aggregated features, which significantly degrades their utility and the model performance. To compensate, GAP concatenates the original features with the noisy ones, so the model mainly learns from the original node features rather than the graph, while the noisy aggregated features hinder learning. If only the noisy features were used, the results would be similar to DPAR. For DPAR, it suffers from **Limitations 1, 2, & 4**. Although it also uses PPR propagation like SaGD, its inferior performance stems from a failure to analyze the sensitivity of the PPR propagation matrix. DPAR uses an approximate computation of the propagation matrix and injects substantial noise into it. As mentioned before, injecting noise into the aggregated features substantially degrades model performance. In contrast, we preserve the exact propagation and derive tight closed-form sensitivity bounds.

DP-GNN suffers from **Limitations 1, 2, & 3**. It largely follows the non-private learning framework but overestimates sensitivity by treating all gradients affected by adding or removing a node as changed, without fine-grained analysis of their actual variation. In contrast, SaGD provides accurate bounds on aggregated features and, for the first time, establishes a direct link between feature sensitivity and gradient sensitivity, going beyond prior methods that rely on artificial clipping. It can be observed that DP-SGD, despite ignoring the graph structure and relying solely on node features, often achieves surprisingly competitive performance. This is because, although it does not use edge information, it avoids the higher sensitivity and added noise introduced by incorporating edges, resulting in more robust model performance.

A comparison between GNN (EW) and GNN (Non-DP) shows that edge weighting (Equation (5)) inevitably degrades performance.

Preserving the original edge weights while ensuring node-DP is fundamentally infeasible, as reflected in the poor performance of DP-GNN. By reducing and re-weighting edges, we substantially lower sensitivity. Although this introduces some utility loss, the resulting reduction in sensitivity more than compensates for it, leading to superior performance over other DP baselines.

Runtime. As stated in Section 4, our algorithm removes edge pruning and per-sample gradient clipping, substantially reducing the training time. We report the average runtime per training epoch and the total runtime on two datasets in Tables 2 and 3, respectively, where one epoch denotes a full pass over the training dataset (iterations = dataset size / batch size). Experiments are conducted on an NVIDIA RTX A5000 GPU. As shown in Table 2, SaGD achieves a training speedup of more than 10× compared to other DP algorithms. This is because existing methods require gradient clipping to bound sensitivity, which involves iterating over all individual gradients and significantly slows training. In contrast, SaGD avoids per-gradient clipping entirely. It mathematically analyzes the gradient computation formula to derive a sensitivity bound and directly adds noise to the gradient sum. During this process, gradients are computed in a fully vectorized manner across the entire batch, leveraging efficient tensor operations rather than iterating over individual samples. This design significantly accelerates training.

For the total runtime of the graph-based methods reported in Table 3, SaGD still achieves over a 10× speedup compared to others. The total runtime of graph-based methods consists of three main parts: edge pruning (if used), message propagation (as described in Section 4.2.2), and model update. The computation of message propagation in SaGD is similar to that of other methods, and so is the time cost. As analyzed in the previous paragraph, SaGD achieves substantial acceleration in the model update stage. In addition, edge pruning is another major source of overhead in existing methods. Regardless of how well it is designed, any edge pruning algorithm must iterate over the edge set E , whose size scales with n^2 , and thus incurs substantial runtime cost. In Table 3, we report the pruning time of existing methods (denoted as Pruning), which accounts for 50 – 90% of their total runtime on large datasets. By eliminating pruning, SaGD avoids this overhead entirely.

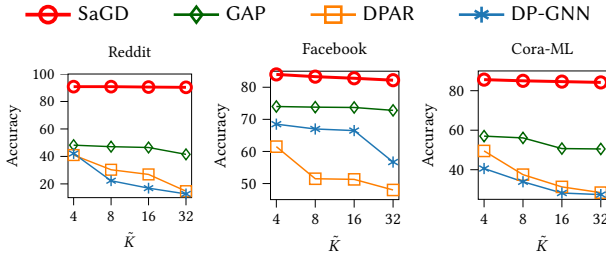
Effect of Node Degree. Here, we assume an ideal edge pruning algorithm that enforces the bounded node degree condition used in prior work and evaluate all graph-based methods under this setting. Specifically, each node is limited to at most $\tilde{K} \in \{4, 8, 16, 32\}$ edges

Table 2: Per epoch runtime comparison (in seconds).

Dataset	SaGD	GAP	DPAR	DP-GNN	DP-SGD
Reddit	0.013	3.8	5.2	11.4	4.0
Facebook	0.004	0.3	0.5	2.2	0.3

Table 3: Total runtime comparison (in seconds).

Dataset	SaGD	GAP	DPAR	DP-GNN	Pruning
Reddit	39.8	7420.9	7485.6	7972.3	7097.4
Facebook	1.2	17.3	19.4	27.9	10.5

**Figure 2: Classification accuracy (in percentage) of the algorithms with varying bounded degree $\tilde{K}$ under $\epsilon = 4$.**

by randomly removing excess edges as in prior studies. Neighboring datasets are assumed to differ by one node and up to $\tilde{K}$ edges. As shown in Figure 2, all methods achieve better performance than in Table 1, since a bounded degree reduces sensitivity and required noise. Meanwhile, our method maintains superior and consistent performance across different $\tilde{K}$ values, as it adaptively adjusts edge weights (Equation 5), whereas the sensitivity of other methods grows polynomially with $\tilde{K}$.

Ablation Study. The PPR propagation scheme has been shown to achieve superior performance compared with GNNs using the original propagation scheme in traditional settings [13]. Here, we conduct an ablation study to evaluate its effectiveness within our decoupled learning framework under both private ($\epsilon = 4$) and non-private ($\epsilon = \infty$) settings. As shown in Table 4, SaGD with PPR propagation consistently outperforms its counterpart with the original propagation, with more improvement in the private setting due to the lower sensitivity of PPR (Lemma 1). Specifically, by aggregating less neighborhood information during the message aggregation stage, PPR reduces the influence of each node on others, which in turn lowers the required noise magnitude while preserving the learning utility. This result highlights the benefit of our sensitivity-aware algorithm design and analysis. In contrast, existing methods typically rely on black-box gradient clipping, which prevents precise characterization of gradient sensitivity changes. Consequently, they cannot effectively leverage propagation schemes such as PPR that explicitly control the amount of propagated information.

Table 4: Comparison of SaGD accuracy (in percentage) with PPR and original propagation schemes under different ϵ .

Dataset	PPR ($\epsilon = 4$)	Orig. ($\epsilon = 4$)	PPR ($\epsilon = \infty$)	Orig. ($\epsilon = \infty$)
Reddit	89.8	85.9	94.6	94.0
Facebook	81.0	78.0	92.8	92.0

Attack Evaluation. SaGD is theoretically resilient to inference attacks by satisfying DP. We further validate this property through a node membership inference attack in Appendix D.

7 Related Work

Graph learning on sensitive node and edge data makes GNNs vulnerable to attribute and link inference attacks [19, 29]. A common defense is differential privacy (DP) [11], which mitigates these risks by bounding sensitivity and injecting noise [1]. However, for graph data, correlations induced by edges pose unique challenges.

Researchers have proposed various DP methods for protecting node-level privacy in graph learning [7, 8, 32, 38, 42]. For example, Daigavane et al. [8] propose a graph neighborhood sampling scheme and extend the privacy amplification technique from DP-SGD to graph learning. GAP[32] and DPAR[42] decouple message propagation from model training. However, due to limitations in their theoretical analysis, they add noise to both the propagation results and the training gradients, whereas our method injects noise only into the training gradients. DPDGC [7] encodes the adjacency matrix as features and trains with DP-SGD, but it relies on a new privacy definition that remains unvalidated. Xiang et al. [38] propose to sample the edges and replace the Gaussian noise in gradient perturbation with symmetric multivariate Laplace (SML) noise. However, as discussed in [2, 39], there were gaps in the analysis in [38] that invalidate the privacy guarantee of the SML mechanism.

Edge-level privacy [24, 35] and decentralized graph learning [31] are related but distinct from our setting.

8 Conclusion

In this paper, we propose SaGD, a rigorous node-level differentially private framework for graph learning. Our contributions include: (i) a decoupled graph learning framework with noise injection only into gradients; (ii) an edge weighting mechanism that preserves the graph topology; (iii) tight, closed-form sensitivity bounds for multi-hop aggregation; and (iv) sensitivity-aware gradient perturbation without clipping, which avoids sensitivity overestimation and the overhead of per-sample gradient clipping. Experiments on real-world graphs show that SaGD achieves superior privacy-utility trade-offs and over 10 $\times$ faster training than existing methods.

Acknowledgments

This research is supported by the National Research Foundation, Singapore under its AI Singapore Programme (AISG Award No: AISG3-RP-2022-029). Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of National Research Foundation, Singapore.

References

- [1] Martin Abadi, Andy Chu, Ian Goodfellow, H Brendan McMahan, Ilya Mironov, Kunal Talwar, and Li Zhang. 2016. Deep learning with differential privacy. In *CCS*. 308–318.
- [2] Authors of [38]. 2025. Personal communications.
- [3] Borja Balle, Gilles Barthe, Marco Gaboardi, Justin Hsu, and Tetsuya Sato. 2020. Hypothesis testing interpretations and renyi differential privacy. In *AISTATS*. PMLR, 2496–2506.
- [4] Jeremiah Blocki, Avrim Blum, Anupam Datta, and Or Sheffet. 2013. Differentially private data analysis of social networks via restricted sensitivity. In *ITCS*. 87–96.
- [5] Sebastian Bruch, Xuanhui Wang, Michael Bendersky, and Marc Najork. 2019. An analysis of the softmax cross entropy loss for learning-to-rank with binary relevance. In *ACM SIGIR*. 75–78.
- [6] Zhiqi Bu, Jialin Mao, and Shiyun Xu. 2022. Scalable and efficient training of large convolutional neural networks with differential privacy. In *NeurIPS*. 38305–38318.
- [7] Eli Chien, Wei-Ning Chen, Chao Pan, Pan Li, Ayfer Özgür, and Olgica Milenkovic. 2023. Differentially private decoupled graph convolutions for multigranular topology protection. In *NeurIPS*. 45381–45401.
- [8] Ameiya Daigavane, Gagan Madan, Aditya Sinha, Abhradeep Guha Thakurta, Gaurav Aggarwal, and Prateek Jain. 2021. Node-Level Differentially Private Graph Neural Networks. In *ICLR Workshop on PAIR2Struct: Privacy, Accountability, Interpretability, Robustness, Reasoning on Structured Data*.
- [9] Wei-Yen Day, Ninghui Li, and Min Lyu. 2016. Publishing graph degree distribution with node differential privacy. In *SIGMOD*. 123–138.
- [10] Wei Dong, Juanru Fang, Ke Yi, Yuchao Tao, and Ashwin Machanavajjhala. 2022. R2t: Instance-optimal truncation for differentially private query evaluation with foreign keys. In *SIGMOD*. 759–772.
- [11] Cynthia Dwork. 2006. Differential privacy. In *ICALP*. 1–12.
- [12] Cynthia Dwork, Aaron Roth, et al. 2014. The algorithmic foundations of differential privacy. *Foundations and Trends® in Theoretical Computer Science* 9, 3–4 (2014), 211–407.
- [13] Johannes Gasteiger, Aleksandar Bojchevski, and Stephan Günnemann. 2019. Predict then Propagate: Graph Neural Networks meet Personalized PageRank. In *ICLR*.
- [14] Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. 2017. Neural message passing for quantum chemistry. In *ICML*. Pmlr, 1263–1272.
- [15] Eric Goldman. 2020. An introduction to the california consumer privacy act (ccpa). *Santa Clara Univ. Legal Studies Research Paper* (2020).
- [16] Mridul Gupta, Samyak Jain, Vansh Raman, HARIPRASAD KODAMANA, and Sayan Ranu. 2025. Bonsai: Gradient-free Graph Condensation for Node Classification. In *ICLR*.
- [17] Richard Hartley and Andrew Zisserman. 2003. *Multiple view geometry in computer vision*. Cambridge university press.
- [18] Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. 2020. LightGCN: Simplifying and powering graph convolution network for recommendation. In *SIGIR*. 639–648.
- [19] Xinlei He, Jinyuan Jia, Michael Backes, Neil Zhenqiang Gong, and Yang Zhang. 2021. Stealing links from graph neural networks. In *USENIX Security*. 2669–2686.
- [20] Chris Jay Hoofnagle, Bart Van Der Sloot, and Frederik Zuiderveen Borgesius. 2019. The European Union general data protection regulation: what it is and what it means. *Information & Communications Technology Law* 28, 1 (2019), 65–98.
- [21] Keke Huang, Wencai Cao, Hoang Ta, Xiaokui Xiao, and Pietro Liò. 2024. Optimizing polynomial graph filters: A novel adaptive krylov subspace approach. In *WWW*. 1057–1068.
- [22] Jinyuan Jia and Neil Zhenqiang Gong. 2018. {AttriGuard}: A practical defense against attribute inference attacks via adversarial machine learning. In *USENIX Security*. 513–529.
- [23] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *ICLR*.
- [24] Aashish Kolluri, Teodora Baluta, Bryan Hooi, and Prateek Saxena. 2022. LPGNet: Link Private Graph Networks for Node Classification. In *CCS*. 1813–1827.
- [25] Xiao Li, Li Sun, Mengjie Ling, and Yan Peng. 2023. A survey of graph neural network based recommendation in social networks. *Neurocomputing* 549 (2023), 126441.
- [26] Yuankai Luo, Lei Shi, and Xiao-Ming Wu. 2024. Classic GNNs are strong baselines: reassessing GNNs for node classification. In *NeurIPS*. 97650–97669.
- [27] Ilya Mironov. 2017. Rényi differential privacy. In *IEEE CSF*. 263–275.
- [28] Diganta Misra. 2019. Mish: A self regularized non-monotonic activation function. *arXiv preprint arXiv:1908.08681* (2019).
- [29] Iyola E Olatunji, Wolfgang Nejdl, and Megha Khosla. 2021. Membership inference attack on graph neural networks. In *TPS-ISA*. IEEE, 11–20.
- [30] Lawrence Page. 1999. *The PageRank citation ranking: Bringing order to the web*. Technical Report. Technical Report.
- [31] Sina Sajadmanesh and Daniel Gatica-Perez. 2021. Locally private graph neural networks. In *CCS*. 2130–2145.
- [32] Sina Sajadmanesh, Ali Shahin Shamsabadi, Aurélien Bellet, and Daniel Gatica-Perez. 2023. GAP: Differentially private graph neural networks with aggregation perturbation. In *USENIX Security*.
- [33] Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. 2017. Membership inference attacks against machine learning models. In *S&P*. IEEE, 3–18.
- [34] Jianxin Wei, Ergute Bao, Xiaokui Xiao, and Yin Yang. 2022. Dpis: An enhanced mechanism for differentially private sgd with importance sampling. In *CCS*. 2885–2899.
- [35] Jianxin Wei, Yizheng Zhu, Xiaokui Xiao, Ergute Bao, Yin Yang, Kuntai Cai, and Beng Chin Ooi. 2025. GCON: Differentially Private Graph Convolutional Network via Objective Perturbation. In *ICDE*.
- [36] Felix Wu, Amauri Souza, Tianyi Zhang, Christopher Fifty, Tao Yu, and Kilian Weinberger. 2019. Simplifying graph convolutional networks. In *ICML*. PMLR, 6861–6871.
- [37] Shiwen Wu, Fei Sun, Wentao Zhang, Xu Xie, and Bin Cui. 2022. Graph neural networks in recommender systems: a survey. *ACM CSUR* 55, 5 (2022), 1–37.
- [38] Zihang Xiang, Tianhao Wang, and Di Wang. 2024. Preserving node-level privacy in graph neural networks. In *S&P*. IEEE, 4714–4732.
- [39] Zihang Xiang, Tianhao Wang, and Di Wang. 2025. Preserving Node-level Privacy in Graph Neural Networks. *arXiv:2311.06888* <https://arxiv.org/abs/2311.06888>
- [40] Renchi Yang, Yidu Wu, Xiaoyang Lin, Qichen Wang, Tsz Nam Chan, and Jieming Shi. 2024. Effective Clustering on Large Attributed Bipartite Graphs. In *KDD*. 3782–3793.
- [41] Ashkan Yousefpour, Igor Shilov, Alexandre Sablayrolles, Davide Testuggine, Karthik Prasad, Mani Malek, John Nguyen, Sayan Ghosh, Akash Bharadwaj, Jessica Zhao, et al. 2021. Opacus: User-Friendly Differential Privacy Library in PyTorch. In *NeurIPS 2021 Workshop Privacy in Machine Learning*.
- [42] Qiuchen Zhang, Hong kyu Lee, Jing Ma, Jian Lou, Carl Yang, and Li Xiong. 2024. DPAR: Decoupled Graph Neural Networks with Node-Level Differential Privacy. In *WWW*. 1170–1181.

A Further Explanation

A.1 An Example of Edge Pruning

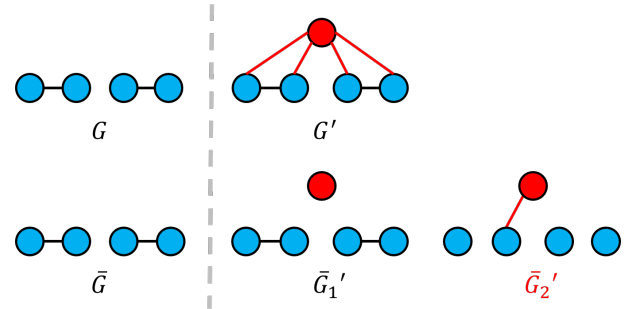


Figure 3: Edge pruning for node-level DP.

For example, as shown in Figure 3, consider a graph G with four nodes, and a graph G' formed by adding a new node connected to all existing nodes. With a pruning threshold $K = 1$, the pruned graph $\bar{G}$ remains unchanged, but G' may be pruned into either $\bar{G}_1$ or $\bar{G}_2$. While $\bar{G}_1$ differs from $\bar{G}$ by at most one edge, $\bar{G}_2$ differs by three edges and all nodes, since every node's neighbors have changed. An example of the pruning procedure in which G' is pruned into $\bar{G}_2$ is provided in the following paragraph. Prior work assumes that $\bar{G}_1$ is the resulting pruned graph by default, where only the added node is affected. However, to satisfy node-DP, one must account for the worst-case scenario $\bar{G}_2$, in which all existing nodes are affected due to the red node in $\bar{G}'$, resulting in $O(|V|)$ sensitivity.

Here is an example corresponding to $G' \rightarrow \bar{G}_2$ in Figure 3. Label the five nodes in Figure 1 from left to right as a, b, c (the red node), d, e . We then consider pruning the edge list $[< d, e >, < c, d >, < c, e >, < a, b >, < c, a >, < c, b >]$ to enforce a maximum degree of 1. For

the first edge $\langle d, e \rangle$, both nodes d and e initially have degree 2, so $\langle d, e \rangle$ is discarded. Next, for $\langle c, d \rangle$ and $\langle c, e \rangle$, although nodes d and e now have degree 1, node c has degree 4, so $\langle c, d \rangle$ and $\langle c, e \rangle$ are discarded. Similarly, $\langle a, b \rangle$ and $\langle c, a \rangle$ are discarded, leaving only the edge $\langle c, b \rangle$ as shown on the right of Figure 1. Although the order of the edge list may differ from that shown above, we must account for the worst-case ordering when ensuring differential privacy. Moreover, the removal strategy should not rely on observing the data, as this would violate privacy. To the best of our knowledge, no node-level, privacy-preserving, or data-independent edge pruning strategy is currently available.

A.2 Diagonal Elements of $\tilde{\mathbf{A}}$

The diagonal elements of $\tilde{\mathbf{A}}$ are positive because

$$A_{ii} = 1 - k_i \cdot \min\left(\frac{\mu}{n}, \frac{1}{k_i + 1}\right) \geq 1 - \frac{k_i}{k_i + 1} = \frac{1}{k_i + 1}.$$

B Sensitivity of Z

PROOF. We first prove the bounds under the element-wise normalization in Equation (5), and then under the row-wise normalization in Equation (2).

For element-wise normalization. We prove the sensitivities of $\tilde{\mathbf{A}}_m$ and $\mathbf{Z}_m$ sequentially. Without loss of generality, we assume that D' is obtained from adding an $n+1$ -th node to D . Let $k_i \in [0, K]$ be the out edges of the node i , $i \in [1, n+1]$. Note that $K \in [1, n]$ denotes the maximum possible node degree. Let the edges of the node $n+1$ connect to nodes $1, 2, \dots, k_{n+1}$. We compute $\tilde{\mathbf{A}}$, $\mathbf{R}_m$, and $\mathbf{Z}_m$ on D and $\tilde{\mathbf{A}}'$, $\mathbf{R}'_m$, and $\mathbf{Z}'_m$ on D' by Equations (5, 4, 6), respectively. For the convenience of subsequent calculations, we pad the $n+1$ -th dimension of $\mathbf{X}$, $\tilde{\mathbf{A}}$, $\mathbf{R}_m$, and $\mathbf{Z}_m$ with zeros (and maintain their notations for simplicity), so that their dimensionalities match those of $\mathbf{X}'$, $\tilde{\mathbf{A}}'$, $\mathbf{R}'_m$, and $\mathbf{Z}'_m$.

$\tilde{\mathbf{A}}$ and $\tilde{\mathbf{A}}'$ differ in rows $1, 2, \dots, k_{n+1}$ and $n+1$. We have

$$\tilde{\mathbf{A}}' - \tilde{\mathbf{A}} = \sum_{i=1}^{k_{n+1}} \mathbf{e}_i \tilde{\mathbf{a}}_i^\top + \mathbf{e}_{n+1} \tilde{\mathbf{a}}_{n+1}^\top, \quad (10)$$

where $\mathbf{e}_i$ is a vector whose i -th element is 1 and all other elements are 0. For $\tilde{\mathbf{a}}_i$ (except $\tilde{\mathbf{a}}_{n+1}$), we consider the nonzero elements in $\tilde{\mathbf{a}}_i$, whose values depend on the ordering between $\frac{\mu}{K}$ and $\frac{1}{k_i+1}$. There are two cases of $\tilde{\mathbf{a}}_i$:

- (1) $\frac{\mu}{K} \geq \frac{1}{k_i+2}$. In this case, $\tilde{\mathbf{a}}_i$ contains $k_i + 1$ nonzero elements with absolute value $\min\left(\frac{\mu}{K}, \frac{1}{k_i+1}\right) - \frac{1}{k_i+2} \leq \frac{1}{(k_i+1)(k_i+2)}$ and one nonzero element with absolute value $\frac{1}{k_i+2}$. Hence, $\|\tilde{\mathbf{a}}_i\|_1 \leq \frac{2}{k_i+2} \leq \frac{2\mu}{K}$.
- (2) $\frac{\mu}{K} < \frac{1}{k_i+2}$. In this case, $\tilde{\mathbf{a}}_i$ contains two elements with absolute value $\frac{\mu}{K}$. Then $\|\tilde{\mathbf{a}}_i\|_1 = \frac{2\mu}{K} < \frac{2}{k_i+1}$.

Therefore,

$$\|\tilde{\mathbf{a}}_i\|_1 \leq \min\left(\frac{2\mu}{K}, \frac{2}{k_i+1}\right). \quad (11)$$

This bound holds regardless of whether $\tilde{\mathbf{A}}'$ is obtained from $\tilde{\mathbf{A}}$ by adding or by removing one node. Then we have

$$\begin{aligned} \psi(\tilde{\mathbf{A}}\mathbf{X}, \tilde{\mathbf{A}}'\mathbf{X}') &= \sum_{i=1}^{k_{n+1}} \|\tilde{\mathbf{a}}_i^\top \mathbf{X}'\|_2 \leq \sum_{i=1}^{k_{n+1}} \|\tilde{\mathbf{a}}_i\|_1 \max_j \|\mathbf{x}_j\|_2 \\ &\leq k_{n+1} \cdot \frac{2\mu}{K} \cdot 1 \leq 2\mu = S(\tilde{\mathbf{A}}\mathbf{X}). \end{aligned} \quad (12)$$

For $\tilde{\mathbf{A}}_m\mathbf{X}$, we have

$$\begin{aligned} \tilde{\mathbf{A}}'_m\mathbf{X}' - \tilde{\mathbf{A}}_m\mathbf{X} &= \tilde{\mathbf{A}}'_m\mathbf{X}' - \tilde{\mathbf{A}}_m\mathbf{X}' \\ &= \tilde{\mathbf{A}}'_{m-1}\tilde{\mathbf{A}}'\mathbf{X}' - \tilde{\mathbf{A}}_{m-1}\tilde{\mathbf{A}}'\mathbf{X}' + \tilde{\mathbf{A}}_{m-1}\tilde{\mathbf{A}}'\mathbf{X}' - \tilde{\mathbf{A}}_{m-1}\tilde{\mathbf{A}}\mathbf{X}' \\ &= (\tilde{\mathbf{A}}'_{m-1} - \tilde{\mathbf{A}}_{m-1})(\tilde{\mathbf{A}}'\mathbf{X}') + (\tilde{\mathbf{A}}_{m-1}\tilde{\mathbf{A}}' - \tilde{\mathbf{A}}_{m-1}\tilde{\mathbf{A}})\mathbf{X}'. \end{aligned} \quad (13)$$

Since every element of $\tilde{\mathbf{A}}'$ is nonnegative and each row sums to 1, the L_2 -norm of each row of $\tilde{\mathbf{A}}'\mathbf{X}'$ is still bound by 1. Hence, $\tilde{\mathbf{A}}'\mathbf{X}'$ can be regarded as some new features that are equivalent to $\mathbf{X}'$ when evaluating $\sup \psi$. We have

$$\begin{aligned} \sup \psi(\tilde{\mathbf{A}}_{m-1}(\tilde{\mathbf{A}}'\mathbf{X}'), \tilde{\mathbf{A}}'_{m-1}(\tilde{\mathbf{A}}'\mathbf{X}')) \\ = \sup \psi(\tilde{\mathbf{A}}_{m-1}\mathbf{X}', \tilde{\mathbf{A}}'_{m-1}\mathbf{X}') = \sup \psi(\tilde{\mathbf{A}}_{m-1}\mathbf{X}, \tilde{\mathbf{A}}'_{m-1}\mathbf{X}'). \end{aligned} \quad (14)$$

By Equations (13, 14),

$$\begin{aligned} \sup \psi(\tilde{\mathbf{A}}_m\mathbf{X}, \tilde{\mathbf{A}}'_m\mathbf{X}') \\ \leq \sup \psi(\tilde{\mathbf{A}}_{m-1}\mathbf{X}, \tilde{\mathbf{A}}'_{m-1}\mathbf{X}') + \sup \psi(\tilde{\mathbf{A}}_{m-1}\tilde{\mathbf{A}}\mathbf{X}', \tilde{\mathbf{A}}_{m-1}\tilde{\mathbf{A}}'\mathbf{X}') \end{aligned} \quad (15)$$

According to Appendix A.2, each element of $\tilde{\mathbf{A}}$ is nonnegative. Hence, each element of $\tilde{\mathbf{A}}_m$ is nonnegative. According to Lemma 1 in [35], for any m , the sum of i -th column of $\tilde{\mathbf{A}}_m$ satisfies

$$\sum_{j=1}^{n+1} \tilde{\mathbf{A}}_{m-1,ji} \leq \max\left((k_i + 1)\frac{\mu}{K}, 1\right) \quad (16)$$

where $\tilde{\mathbf{A}}_{m-1,ji}$ is the (j, i) -th element of $\tilde{\mathbf{A}}_{m-1}$. Combining Equations (11, 16), we have

$$\begin{aligned} \psi(\tilde{\mathbf{A}}_{m-1}\tilde{\mathbf{A}}\mathbf{X}', \tilde{\mathbf{A}}_{m-1}\tilde{\mathbf{A}}'\mathbf{X}') &\leq \sum_{i=1}^{k_{n+1}} \sum_{j=1}^{n+1} \tilde{\mathbf{A}}_{m-1,ji} \|\tilde{\mathbf{a}}_i^\top \mathbf{X}'\|_2 \\ &\leq \sum_{i=1}^{k_{n+1}} \sum_{j=1}^{n+1} \tilde{\mathbf{A}}_{m-1,ji} \|\tilde{\mathbf{a}}_i\|_1 \max_j \|\mathbf{x}_j\|_2 \\ &\leq \sum_{i=1}^{k_{n+1}} \max\left(\frac{\mu(k_i + 1)}{K}, 1\right) \min\left(\frac{2\mu}{K}, \frac{2}{k_i + 1}\right) = k_{n+1} \cdot \frac{2\mu}{K} \leq 2\mu. \end{aligned} \quad (17)$$

Hence, by Equations (12, 15, 17),

$$\psi(\tilde{\mathbf{A}}_m\mathbf{X}, \tilde{\mathbf{A}}'_m\mathbf{X}') \leq 2\mu m = S(\tilde{\mathbf{A}}_m\mathbf{X}),$$

$$\begin{aligned} \psi(\mathbf{Z}_m, \mathbf{Z}'_m) &\leq \gamma \sum_{i=0}^{m-1} (1 - \gamma)^i S(\tilde{\mathbf{A}}_i\mathbf{X}) + (1 - \gamma)^m S(\tilde{\mathbf{A}}_m\mathbf{X}) \\ &= \frac{2\mu(1 - \gamma)}{\gamma} [1 - (1 - \gamma)^m] = S(\mathbf{Z}_m). \end{aligned}$$

For row-wise normalization. Equations (13, 14, 15) still holds. We define $\bar{\mathbf{a}}_i$ in the same way as Equation (10). $\bar{\mathbf{a}}_i$ contains two elements with absolute value at most $\frac{1}{2}$. Hence, $\psi(\bar{\mathbf{A}}\mathbf{X}, \bar{\mathbf{A}}'\mathbf{X}')$ should be modified to

$$\psi(\bar{\mathbf{A}}\mathbf{X}, \bar{\mathbf{A}}'\mathbf{X}') \leq k_{n+1} \cdot 2 \cdot \frac{1}{2} \leq K = S(\bar{\mathbf{A}}\mathbf{X}).$$

Similarly, we have $\psi(\bar{\mathbf{A}}_{m-1}\bar{\mathbf{A}}\mathbf{X}', \bar{\mathbf{A}}_{m-1}\bar{\mathbf{A}}'\mathbf{X}') \leq K$. Hence, combining with Equation (15), we have

$$\psi(\bar{\mathbf{A}}_m\mathbf{X}, \bar{\mathbf{A}}'_m\mathbf{X}') \leq Km = S(\bar{\mathbf{A}}_m\mathbf{X}),$$

and

$$\begin{aligned} \psi(\bar{\mathbf{Z}}_m, \bar{\mathbf{Z}}'_m) &\leq \gamma \sum_{i=0}^{m-1} (1-\gamma)^i S(\bar{\mathbf{A}}_i\mathbf{X}) + (1-\gamma)^m S(\bar{\mathbf{A}}_m\mathbf{X}) \\ &= \frac{K(1-\gamma)}{\gamma} [1 - (1-\gamma)^m] = S(\bar{\mathbf{Z}}_m). \end{aligned}$$

□

C Sensitivity of g

C.1 Lemma 2

PROOF. We illustrate the proof with a two-layer network model. Without loss of generality, assume D' is obtained by adding the $(n+1)$ -th node to D . Let $\mathbf{f}_i = \frac{\partial f(\hat{\mathbf{y}}_i, \mathbf{y}_i)}{\partial \hat{\mathbf{y}}_i}$ and $\mathbf{f}'_i = \frac{\partial f(\hat{\mathbf{y}}'_i, \mathbf{y}_i)}{\partial \hat{\mathbf{y}}'_i}$. Let $\mathbf{h}_i = \frac{\partial h(\mathbf{\Theta}_1 \mathbf{z}_i)}{\partial \mathbf{\Theta}_1}$ and $\mathbf{h}'_i = \frac{\partial h(\mathbf{\Theta}_1 \mathbf{z}'_i)}{\partial \mathbf{\Theta}_1}$. The gradients of the layers with respect to the loss $f(\hat{\mathbf{y}}_i, \mathbf{y}_i)$ is

$$g(\mathbf{\Theta}_1, \hat{\mathbf{y}}_i) = \text{diag}(\mathbf{h}_i) \mathbf{\Theta}_2^\top \mathbf{f}_i \mathbf{z}_i^\top, \quad g(\mathbf{\Theta}_2, \hat{\mathbf{y}}_i) = \mathbf{f}_i (h(\mathbf{\Theta}_1 \mathbf{z}_i))^\top.$$

For the activations, we have $\|h(x)\|_2 \leq \|x\|_2$. Note that $\|\mathbf{x}_i\|_2 \leq 1$ for any $i \in [1, n+1]$ and the sum row of $\mathbf{R}_m$ is always 1. Hence, $\|\mathbf{z}_i\|_2 \leq 1$ for any $i \in [1, n+1]$. For the $(n+1)$ -th node, we have

$$\begin{aligned} \|g(\mathbf{\Theta}_1, \hat{\mathbf{y}}_i)\|_2 &\leq c_1 c_{h1} \theta_2 = S(g_{\text{new}}(\mathbf{\Theta}_1)), \\ \|g(\mathbf{\Theta}_2, \hat{\mathbf{y}}_i)\|_2 &\leq c_1 \theta_1 = S(g_{\text{rest}}(\mathbf{\Theta}_2)). \end{aligned} \quad (18)$$

The specific values of the Lipschitz constants used in this work, including c_{h1} , c_{h2} , c_1 , and c_2 , are provided in Appendix C.2.

For any of the other nodes, we have

$$\begin{aligned} &\|g(\mathbf{\Theta}_1, \hat{\mathbf{y}}_i) - g(\mathbf{\Theta}_1, \hat{\mathbf{y}}'_i)\|_2 \\ &= \left\| \text{diag}(\mathbf{h}'_i) \mathbf{\Theta}_2^\top [(\mathbf{f}_i - \mathbf{f}'_i) \mathbf{z}_i^\top + \mathbf{f}'_i (\mathbf{z}_i - \mathbf{z}'_i)^\top] \right. \\ &\quad \left. + (\text{diag}(\mathbf{h}_i) - \text{diag}(\mathbf{h}'_i)) \mathbf{\Theta}_2^\top \mathbf{f}_i \mathbf{z}_i^\top \right\|_2. \end{aligned}$$

For $\|(\mathbf{f}_i - \mathbf{f}'_i)\|_2$, we have

$$\begin{aligned} \|(\mathbf{f}_i - \mathbf{f}'_i)\|_2 &\leq c_2 \|\mathbf{y}_i - \mathbf{y}'_i\|_2 \leq c_2 \theta_2 \|h(\mathbf{\Theta}_1 \mathbf{z}_i) - h(\mathbf{\Theta}_1 \mathbf{z}'_i)\|_2 \\ &\leq c_2 \theta_2 c_{h1} \|\mathbf{\Theta}_1 \mathbf{z}_i - \mathbf{\Theta}_1 \mathbf{z}'_i\|_2 \leq c_2 c_{h1} \theta_2 \theta_1 \|\mathbf{z}_i - \mathbf{z}'_i\|_2 \end{aligned}$$

Then we have

$$\|(\mathbf{f}_i - \mathbf{f}'_i) \mathbf{z}_i^\top + \mathbf{f}'_i (\mathbf{z}_i - \mathbf{z}'_i)^\top\|_2 \leq (c_2 c_{h1} \theta_2 \theta_1 + c_1) \|\mathbf{z}_i - \mathbf{z}'_i\|_2. \quad (19)$$

For $\text{diag}(\mathbf{h}_i) - \text{diag}(\mathbf{h}'_i)$, we have

$$\|\text{diag}(\mathbf{h}_i) - \text{diag}(\mathbf{h}'_i)\|_2 \leq c_{h2} \|\mathbf{\Theta}_1 \mathbf{z}_i - \mathbf{\Theta}_1 \mathbf{z}'_i\|_2 \leq c_{h2} \theta_1 \|\mathbf{z}_i - \mathbf{z}'_i\|_2. \quad (20)$$

Combining Equations (19) and (20),

$$\begin{aligned} &\|g(\mathbf{\Theta}_1, \hat{\mathbf{y}}_i) - g(\mathbf{\Theta}_1, \hat{\mathbf{y}}'_i)\|_2 \\ &= c_{h1} \theta_2 (c_2 c_{h1} \theta_2 \theta_1 + c_1) \|\mathbf{z}_i - \mathbf{z}'_i\|_2 + c_1 \theta_2 c_{h2} \theta_1 \|\mathbf{z}_i - \mathbf{z}'_i\|_2 \\ &= (c_2 c_{h1}^2 \theta_2^2 \theta_1 + c_1 c_{h2} \theta_2 + c_1 c_{h2} \theta_2 \theta_1) \|\mathbf{z}_i - \mathbf{z}'_i\|_2. \end{aligned} \quad (21)$$

Hence,

$$\begin{aligned} &\left\| \sum_{i=1}^n g(\mathbf{\Theta}_1, \hat{\mathbf{y}}_i) - \sum_{i=1}^n g(\mathbf{\Theta}_1, \hat{\mathbf{y}}'_i) \right\|_2 \\ &\leq (c_2 c_{h1}^2 \theta_2^2 \theta_1 + c_1 c_{h2} \theta_2 + c_1 c_{h2} \theta_2 \theta_1) S(\mathbf{Z}) = S(g_{\text{rest}}(\mathbf{\Theta}_1)). \end{aligned} \quad (22)$$

Similarly,

$$S(g_{\text{rest}}(\mathbf{\Theta}_2)) = (c_2 c_{h1} \theta_2 \theta_1^2 + c_1 c_{h1} \theta_1) S(\mathbf{Z}). \quad (23)$$

By Lemmas 3 & 4 and Equations (18, 22, 23), Lemma 2 holds. □

LEMMA 3. The sensitivity of $\mathbf{g}$ satisfies $S(\mathbf{g}) \leq \sqrt{\sum_{l=1}^\omega S(g(\mathbf{\Theta}_l))^2}$, where $S(g(\mathbf{\Theta}_l))$ denotes the sensitivity of the gradient on $\mathbf{\Theta}_l$.

PROOF. For two gradients $\mathbf{g}$ and $\mathbf{g}'$ computed on two neighboring datasets D and D' , we have $\|\mathbf{g} - \mathbf{g}'\|_2^2 = \sum_{l=1}^\omega \|g(\mathbf{\Theta}_l) - g(\mathbf{\Theta}_l)'\|_2^2$. For $g(\mathbf{\Theta}_l)$, $l \in [1, \omega]$, we have $S(g(\mathbf{\Theta}_l)) = \max_{D, D'} \|g(\mathbf{\Theta}_l) - g(\mathbf{\Theta}_l)'\|_2$. Hence, $S(\mathbf{g}) = \max_{D, D'} \|\mathbf{g} - \mathbf{g}'\|_2 \leq \sqrt{\sum_{l=1}^\omega S(g(\mathbf{\Theta}_l))^2}$. □

LEMMA 4. For $\forall l \in [1, \omega]$, $S(g(\mathbf{\Theta}_l)) \leq S(g_{\text{new}}(\mathbf{\Theta}_l)) + S(g_{\text{rest}}(\mathbf{\Theta}_l))$.

PROOF. Without loss of generality, let D' be the dataset obtained by adding the $(n+1)$ -th node to D . For gradients $g(\mathbf{\Theta}_l)$ and $g(\mathbf{\Theta}_l)'$ on these datasets, we have

$$\begin{aligned} S(g(\mathbf{\Theta}_l)) &= \max_{D, D'} \|g(\mathbf{\Theta}_l) - g(\mathbf{\Theta}_l)'\|_2 \\ &= \max_{D, D'} \left\| \sum_{i=1}^n g(\mathbf{\Theta}_l, \hat{\mathbf{y}}_i) - \sum_{i=1}^{n+1} g(\mathbf{\Theta}_l, \hat{\mathbf{y}}'_i) \right\|_2 \\ &\leq \max_{D, D'} \left\| \sum_{i=1}^n g(\mathbf{\Theta}_l, \hat{\mathbf{y}}_i) - \sum_{i=1}^n g(\mathbf{\Theta}_l, \hat{\mathbf{y}}'_i) \right\|_2 + \|g(\mathbf{\Theta}_l, \hat{\mathbf{y}}'_{n+1})\|_2 \\ &= \max_{D, D'} (g_{\text{rest}}(\mathbf{\Theta}_l) + g_{\text{new}}(\mathbf{\Theta}_l)) \leq S(g_{\text{new}}(\mathbf{\Theta}_l)) + S(g_{\text{rest}}(\mathbf{\Theta}_l)). \end{aligned}$$

□

C.2 Lipschitz Constant of the Loss

For activation functions, which are applied element-wise (univariate), the Lipschitz constants can be obtained via differentiation and typically take values close to 1. In this work, we consider the GELU and Mish activations. For GELU, we have $c_{h1} \approx 1.13$ and $c_{h2} \approx 0.80$. For Mish, we have $c_{h1} \approx 1.09$ and $c_{h2} \approx 0.64$.

Let $\hat{\mathbf{y}} = (\hat{y}_1, \dots, \hat{y}_c)$ and $\mathbf{y} = (y_1, \dots, y_c)$ represent the prediction and the one-hot encoded true label. For loss functions, we consider the softmax cross-entropy loss and the pseudo-Huber loss

$$f(\hat{\mathbf{y}}, \mathbf{y}) = \frac{\rho^2}{c} \sum_{i=1}^c \left(\sqrt{1 + \left(\frac{y_i - \hat{y}_i}{\rho} \right)^2} - 1 \right), \quad (24)$$

where ρ is a hyperparameter. Their Lipschitz constants are given in Lemma 5.

LEMMA 5. For softmax cross-entropy loss [5], we have

$$c_1 \leq \sqrt{2}, \quad c_2 \leq \frac{\sqrt{c-1}}{c}.$$

For pseudo-huber loss as shown in Equation (24), we have

$$c_1 \leq \frac{\rho}{\sqrt{c}}, \quad c_2 \leq \frac{1}{\sqrt{c}}.$$

□

PROOF. For the softmax cross-entropy loss, let $\mathbf{p} = (p_1, \dots, p_c)$ be a vector where $p_i = \frac{\exp(\hat{y}_i)}{\sum_{j=1}^c \exp(\hat{y}_j)}$. We have

$$\frac{\partial f(\hat{\mathbf{y}}, \mathbf{y})}{\partial \hat{y}_i} = p_i - y_i \quad \text{and} \quad J_{ij} = \begin{cases} p_i(1 - p_i), & i = j, \\ -p_i p_j, & i \neq j. \end{cases}$$

Let l denote the index corresponding to the class of $\mathbf{y}$, such that $y_l = 1$. Then we have,

$$\left\| \frac{\partial f(\hat{\mathbf{y}}, \mathbf{y})}{\partial \hat{\mathbf{y}}} \right\|_2^2 = \sum_{i=1}^c (p_i - y_i)^2 = (p_l - 1)^2 + \sum_{i=1, i \neq l}^c p_i^2 \leq 2$$

and

$$\|\mathbf{J}\|_2^2 = \sum_{i=1}^c p_i^2 (1 - p_i)^2 + \sum_{i=1}^c \sum_{j=1, j \neq i}^c p_i^2 p_j^2 \leq \frac{c-1}{c^2}.$$

Hence, $c_1 \leq \sqrt{2}$ and $c_2 \leq \frac{\sqrt{c-1}}{c}$.

For the pseudo-Huber loss, we have We have

$$\frac{\partial f(\hat{\mathbf{y}}, \mathbf{y})}{\partial \hat{y}_i} = \frac{\hat{y}_i - y_i}{c \sqrt{1 + (\frac{\hat{y}_i - y_i}{\rho^2})^2}} \quad \text{and} \quad J_{ij} = \begin{cases} \frac{1}{c} (1 + (\frac{\hat{y}_i - y_i}{\rho^2})^2)^{-\frac{3}{2}}, & i = j, \\ 0, & i \neq j. \end{cases}$$

Then we have,

$$\begin{aligned} \left\| \frac{\partial f(\hat{\mathbf{y}}, \mathbf{y})}{\partial \hat{\mathbf{y}}} \right\|_2^2 &= \sum_{i=1}^c \frac{(\hat{y}_i - y_i)^2}{c^2 (1 + \frac{(\hat{y}_i - y_i)^2}{\rho^2})} \\ &\leq \sum_{i=1}^c \frac{1}{c^2 (\frac{1}{(\hat{y}_i - y_i)^2} + \frac{1}{\rho^2})} \leq \frac{\rho^2}{c} \end{aligned}$$

and

$$\|\mathbf{J}\|_2^2 = \sum_{i=1}^c \frac{1}{c^2 (1 + \frac{(\hat{y}_i - y_i)^2}{\rho^2})^3} \leq \frac{c}{c^2} = \frac{1}{c}.$$

Hence, $c_1 = \frac{\rho}{\sqrt{c}}$ and $c_2 = \frac{1}{\sqrt{c}}$.

□

C.3 Proof of Theorem 1

PROOF. Algorithm 1 includes two noise mechanisms that consume the privacy budget: one for n and one for the gradient of the network. Since we have the gradient sensitivity bounded by $S(\mathbf{g})$, according to Proposition 7 in [27], releasing the gradient in each iteration of the main network consumes a privacy budget of $(\alpha, \frac{\alpha}{2\sigma_T^2})$ -RDP. Based on the composition theorem of RDP in Proposition 1 in [27], the training of the main network with T iterations satisfies $(\alpha, \frac{t_T \alpha}{2\sigma_T^2})$ -RDP. Similarly, the noise mechanism on n satisfies $(\alpha, \frac{\alpha}{2\sigma_n^2})$ -RDP. Hence, by the composition theorem, Theorem 1 holds. □

D Additional Experiment Settings

Table 5: Statistics of the datasets.

Dataset	Nodes	Edges	Features	Classes
Reddit	232,965	114,615,892	602	41
Facebook	22,470	342,004	128	4
Cora-ML	2995	16,316	2,879	7
PubMed	19,717	88,648	500	3

Hyperparameters. For constructing the message propagation matrix, we tune μ (in Equation (5)) in $\{1, 2\}$, the restart probability γ in $\{0.1, 0.9\}$, and the propagation step m in $\{1, 2\}$. For the prediction network, we tune the number of layers ω in $\{1, 2\}$. We do not use regularization on network parameters. We tune the training epochs t_T in $\{200, 400, 800\}$ and the learning rate η in $\{0.02, 0.1\}$. For the noise multiplier, we fix $\sigma_n = 20$ and select the smallest σ_T to satisfy the given privacy budget.

For tasks on Cora-ML and PubMed, we use a feature encoder (a two-layer MLP trained with DP-SGD using half of the privacy budget) to preprocess node features, following [32]. For the encoder, we use a batch size of 256, train for 200 iterations, and apply a clipping norm of 0.1.

Table 6: Attack resilience under $\epsilon = 4$.

Dataset	SaGD	GAP	DPAR	DP-GNN	DP-SGD	Non-DP
Reddit	0.50	0.51	0.50	0.50	0.51	0.99
Facebook	0.50	0.50	0.50	0.50	0.50	0.80

Attack Evaluation. Since our method satisfies DP, it is theoretically resilient to inference attacks. We further validate this by conducting a node membership inference attack following [29, 32]. The attacker is assumed to know 20% of the nodes used in training and 20% of those not used in training, and exploits differences in classification output probabilities between training and non-training nodes. Table 6 reports the mean AUC of the attack on the compared methods across two datasets under $\epsilon = 4$. An AUC close to 0.50 indicates that the attack is unsuccessful (equivalent to random guessing), while an AUC close to 1 indicates that the attack is highly successful. As shown in Table 6, our method achieves resilience comparable to other methods, effectively mitigating node membership inference attacks. Taken together with Table 1, these results demonstrate that our method achieves substantially higher utility while still maintaining strong resilience to such attacks.